Agentic AI risks are here new data reveals gaps in control visibility and security failures Register for the free May 19 webinar →

Understanding Security Risks in AI-Generated Code

Published 07/09/2025

Written by Andrew Stiefel, Endor Labs.

AI coding assistants are changing the game for developers. They offer speed, convenience, and a way to fill knowledge gaps for busy engineering teams. With just a few lines of natural language, developers can generate entire functions, scripts, or configurations—often faster than they could write them from scratch.

But what's convenient for developers can quietly create chaos for security teams.

A recent study found that 62% of AI-generated code solutions contain design flaws or known security vulnerabilities, even when developers used the latest foundational AI models. The root problem is that AI coding assistants don't inherently understand your application's risk model, internal standards, or threat landscape. That disconnect introduces systemic risks—not just insecure lines of code, but logic flaws, missing controls, and inconsistent patterns that erode trust and security over time.

Understanding how AI coding assistants introduce risk is the first step toward governing their safe use.

Support

Risk #1: Repetition of insecure patterns from training data

Today's foundational large language models (LLMs) train on the vast ecosystem of open source code, learning by pattern matching against them. If an unsafe pattern—such as string-concatenated SQL queries—appears frequently in the training set, the assistant will readily produce it. (And it does — SQL injection is one of the leading causes of vulnerabilities).

As a result, if a developer asks the assistant to "query the users table by ID," the model might return a textbook SQL injection flaw because that pattern appeared thousands of times in GitHub repos:

```
sql = "SELECT * FROM users WHERE id = " + user_input
```

Risk #2: Optimization shortcuts that ignore security context

When prompts are ambiguous, LLMs optimize for the shortest path to a passing result, even if that means using overly powerful or dangerous functions. The model isn't incentivized to reason securely—it's rewarded for solving the task. That often leads to shortcuts that work, but open up critical security issues.

Take the case of evaluating a user-provided math expression. A model might quickly respond with `eval(expression)`, since that one-liner solves the problem. But it also opens the door to remote code execution.

Risk #3: Omission of necessary security controls

In a similar vein, many vulnerabilities aren't the result of developers (or AI assistants) writing "bad code." They come from missing protections like validation steps, access checks, or output encoding that help prevent common weaknesses. AI can unintentionally leave guardrails out because it's unaware of the risk model behind the code.

API endpoints can be especially tricky since they accept user input. An AI coding assistant will deliver an endpoint that accepts input without validating, sanitizing, or authorizing the payload simply because the prompt never said it needed to.

Risk #4: Introduction of subtle logic errors

Of course, some of the most dangerous AI-generated flaws don't look like flaws at all. They emerge in the cracks between logic, edge cases, and business context. They are also harder to spot since the code looks correct and might even pass some basic checks.

Depending on their tuning, AI coding assistants can bias more or less towards novel responses. This can improve output, but it also means they are more likely to re-write code when providing suggestions. For example, an AI assistant might swap a function checking a user's role using `if user.role == "admin"` instead of `if "admin" in user.roles`. The code works for single-role users but fails when users have multiple roles, leading to overly permissive access in some workflows.

How to stay ahead

AI coding assistants are powerful, but they aren't security tools. The best way to use them safely is to augment your development process with the right checks and balances:

1. **Train developers on secure prompting.** Prompts are now the code design specification. Teach developers not just how to use AI coding assistants, but how to guide them with specificity.
2. **Integrate security feedback earlier.** Waiting until the pull request or CI pipeline is too late, so work with developers to give AI coding assistants security insights using an MCP server.
3. **Support secure code reviews.** Human reviewers are still the most important check, but as reviews increase, so does fatigue. Consider ways to augment secure code reviews.

AI coding assistants are changing how vulnerabilities enter your codebase by introducing untrusted code into your environment. Security and engineering teams must work together to catch risks earlier.

Further Reading

- Security and Quality in LLM-Generated Code: A Multi-Language, Multi-Model Analysis
- A Comprehensive Study of LLM Secure Code Generation
- Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs
- Artificial-Intelligence Generated Code Considered Harmful: A Road Map for Secure and High-Quality Code Generation
- A Deep Dive Into Large Language Model Code Generation Mistakes: What and Why?

Artificial Intelligence DevSecOps Vulnerabilities

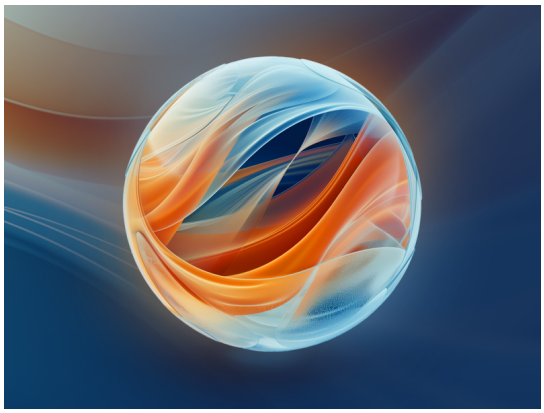
Share this content on your favorite social network today!

Future Cloud

Related Resources

Certificate of Competence in Zero Trust (CCZT)

The industry's first Zero Trust certificate program.



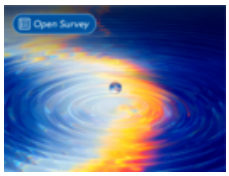
AI Organizational Responsibilities

A blueprint for enterprises to secure AI development and deployment.

AI Safety Initiative

Addressing present and future challenges of AI safety and security.

Latest from CSA



Take the State of Hybrid and Multi-Cloud Security Policy Management Survey

Free Virtual Summit Registration + 50% Off TAISE for Attendees

Webinar: Emerging Reference Patterns for AI Agents and MCP in the Enterprise


CLOUD SECURITY ALLIANCE
RESEARCH REPORT
The Rise in Unstructured Data and AI Security Risks
Build a more resilient foundation for the future of data protection.
Get Your Copy Today!
THALES
CYBERSECURITY
CSA
THALES
Survey Report
The Rise in Unstructured Data and AI Security Risks

Unlock Cloud Security Insights

Email Address Sign up

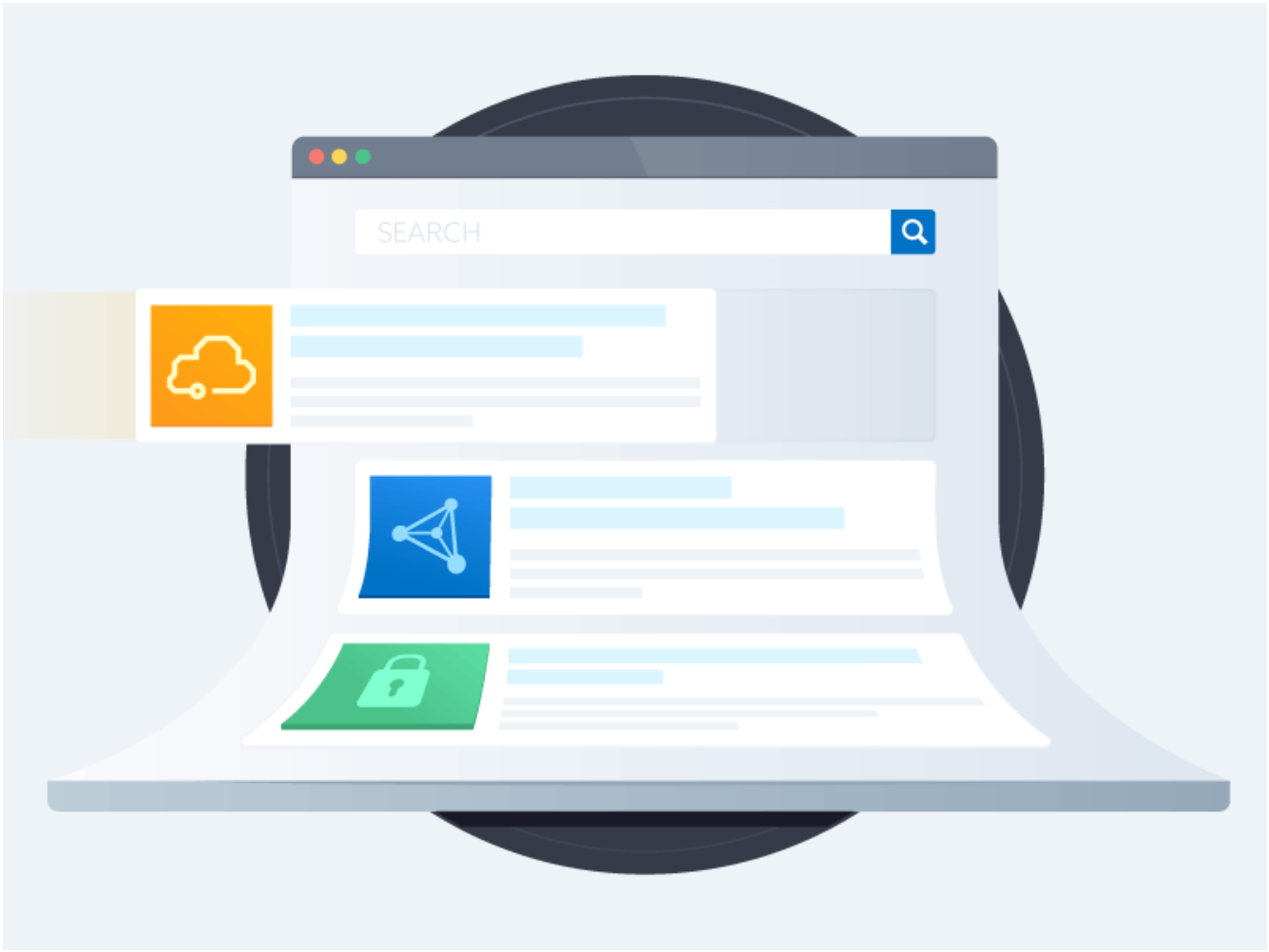
Subscribe to our newsletter for the latest expert trends and updates

Level up with Cloud Infrastructure Security Training

Related Articles:

Patching Faster is Not the Answer to Mythos. Patching Smarter Is.

Published: 05/14/2026



Globee® Awards for Artificial Intelligence (AI) Honors Cloud Security Alliance for AI Leadership with Dual Awards

Published: 05/14/2026

What an AI Lab's Test Reveals About the Enterprise AI Challenge

Published: 05/13/2026

AI Agent Security Starts with Scope Control

Published: 05/12/2026

© 2009–2026 Cloud Security Alliance.
All rights reserved.

Press Coverage
Quality Policy

Our Team

Board of Directors
Management & Staff
Careers

Legal

Privacy Notice
Terms & Conditions

Cloud Security Glossary